2026 第九届传智杯全国大学生数智创新与 AI 应用大赛

# HybridMind

## 作品介绍文档

## 论文精读问答与证据锚点

---


赛道　AI 大模型创新应用挑战赛（技术应用赛 • 个人赛）

组别　B 组（普通本科）

参赛者　张云迪

所在院校　商洛学院


在线演示　https://hanyue0-hybridmind.ms.show

代码仓库　https://github.com/hanyue0/hybridmind

本文档对应报名表中「作品介绍文档」的七项要求

# 一、项目详情及使用方式

## 1.1 这个项目解决什么问题

读英文论文时，真正让人卡住的往往不是通读，而是回答一个具体问题：这篇在哪个数据集上评的？报告的 F1 是多少？作者为什么选 A 而不是 B？把这些丢给通用大模型，几秒钟就能得到一段很像样的回答——但论文场景对"像"的容忍度极低。把 44.1 记成 41.4、把 BART 说成 T5、把测试集数字说成验证集数字，读者若不翻原文几乎无法察觉，而这些数字往往要写进综述或实验对比表。

更麻烦的是核对成本。读者的真实流程是"先问 AI，再翻原文核对"；如果每次都要核对，AI 省下的时间又还回去一半。所以本作品的出发点不是"让 AI 答得更像人"，而是让答案可以被立刻核对：每一条结论后面挂一个锚点，点一下跳到原文那一页、高亮那一行。

HybridMind 就是这样一个中文论文问答系统：上传 PDF → 用中文提问 → 得到答案 + 可点击的原文锚点。它同时解决了一个现实约束：不是每个问题都值得付一次云端费用。系统先让本地 1.5B 小模型作答，再用"答案能否被原文背书"来校验；站得住就留在本地（零 API 费用、数据不出机），站不住才升级云端大模型。

## 1.2 怎么用（在线演示）

打开 https://hanyue0-hybridmind.ms.show/paper，无需注册与配置：

1. 左上角下拉选择一篇论文（内置 3 篇公开论文，也可以拖入自己的 PDF）；
2. 在右侧输入框用中文提问，例如「RAG 使用多少篇 Wikipedia 文档？」；
3. 答案逐字流式出现，同时右侧显示：路由决策（本地/云端、判据）、检索特征、证据锚点；
4. 点击答案里的引用角标或下方锚点条目，左侧原文自动翻页，并把命中的那几行框出来；
5. 底部账本实时累计请求数、云端占比与花费。

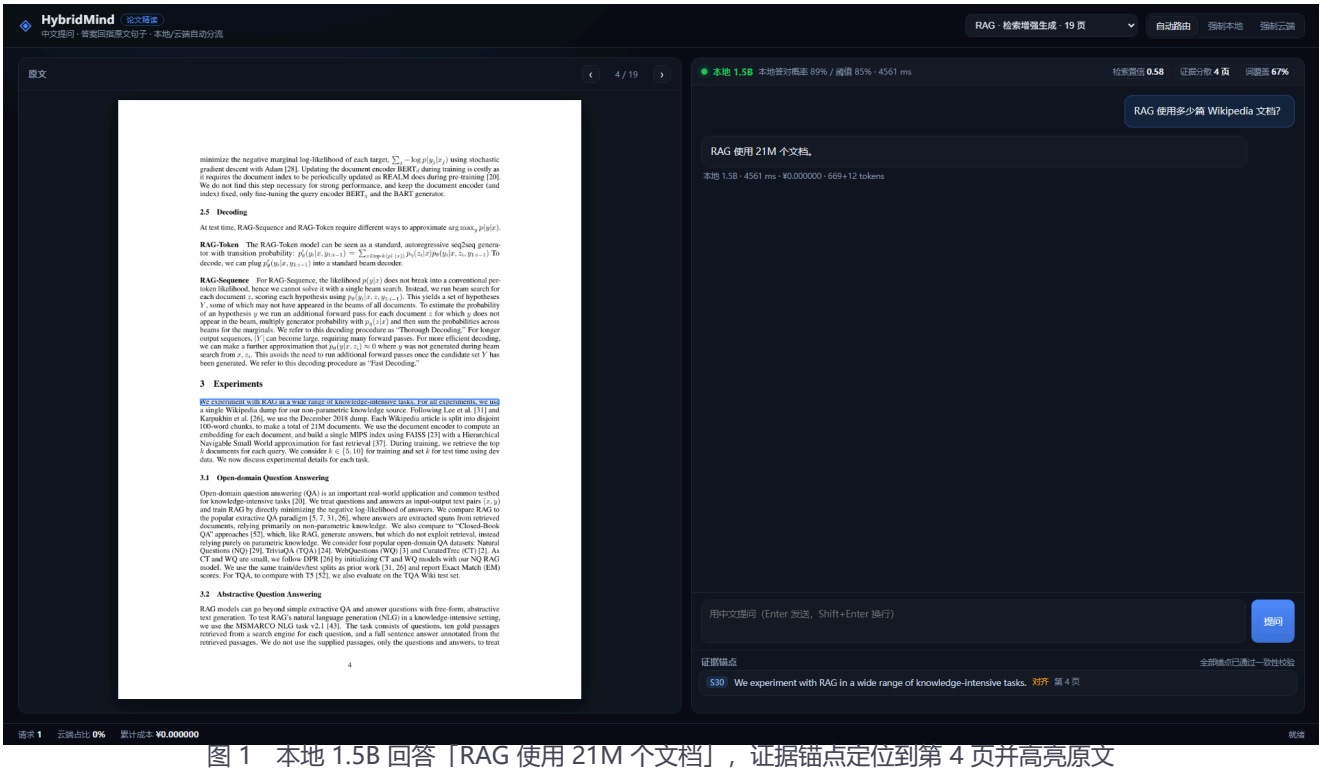



图 1　本地 1.5B 回答「RAG 使用 21M 个文档」，证据锚点定位到第 4 页并高亮原文

## 1.3 怎么在本地跑起来

```
python -m venv .venv && .venv\Scripts\pip install -r requirements.txt
ollama pull qwen2.5:1.5b
copy .env.example .env          # 填 CLOUD_API_KEY（不填则纯本地运行）
python scripts/fetch_papers.py          # 下载 3 篇演示论文
python scripts/build_paper_index.py    # 解析 + 建中文检索索引（约 ¥0.4）
python -m uvicorn app.web.main:app --port 8000    # 打开 /paper
```

没有独立显卡也能跑：本作品在一台 i5-13420H / 16 GB 单通道 / 无独显的笔记本上完成开发与全部评测，本地推理走 CPU（约 3–4 token/s）。

# 二、项目亮点

## 2.1 答案能指回原文的那一行

系统在解析 PDF 时就保留了每个片段的页码与行级坐标，所以前端画出的高亮框可以收紧到命中的那几行，而不是框住半页。答案里的 `[S30]` 是可点击的，点了就跳页并高亮——"可核对"不是文档里的承诺，而是界面上一个能按的按钮。

## 2.2 不靠模型守规矩：确定性对齐

原本的设计是要求模型在回答里标注出处。实测发现：云端模型基本会标，本地 1.5B 经常不标——它把话说圆了，但不写编号。如果锚点层只会解析编号，本地答案就永远没有锚点，整个"先在本地答"的设计会直接失效。

因此增加了一层确定性对齐：用数字（中英文写法一致）、英文术语（直接词面匹配）、译文重合度（建库期的中文译文）三类与语言无关的信号，把答案句对齐到片段；再在片段内部挑出含命中数字/术语最多的句子作为引文。这样模型标不标出处，都能给出可高亮的锚点。

## 2.3 路由信号：从"猜难度"换成"验证据"

本作品前一版本用"预测本地会不会答错"来做路由，在算术题上 AUC 0.9355，效果不错。原样搬到论文场景后，AUC 只有 0.61，阈值化后 93% 的请求都要上云——路由等于失效（详见技术文档 6.4 节）。

原因不是实现错了，而是错误结构变了：算术题里本地模型错在"多步推理保持不住中间结果"，这件事与题目写法强相关；论文题里它错在"在上下文里挑错了那个数"，与题目写法无关。于是我们把路由判据换成一个可验证的信号：本地答完之后，检查答案有没有原文背书（有锚点、引用编号有效、数字可溯），站得住就保留，站不住才升级。

## 2.4 敢报告对自己不利的结果

文档里公开了三件"不好看"的事：其一，本地模型在论文场景只有 63.2% 的准确率，且错误无法被事先预测（AUC 0.61）；其二，门控路由比纯云端少对了 1 道题；其三，判分口径曾经有 bug（"2016年12月"被判成 12），修正后所有历史结果都离线重判并注明。我们认为这些比"再刷高一个点"更有价值——它们决定了读者能不能信任其余的数字。

# 三、AI 能力的测试与评估落地情况

评测集为 68 道中文题，覆盖 3 篇公开论文（63 页、328 个解析片段），题型包括数字定位 34 道、事实抽取 28 道、以及 6 道"原文没有答案"的拒答题。每道题都标注了标准答案、gold page 与来源片段；答案必须在原文里逐字出现，否则整条丢弃。

| 策略 | 准确率 | 证据召回 | 引用有效 | 误拒率 | 云端占比 | 千次成本 |
|---|---|---|---|---|---|---|
| 全本地 1.5B | 63.2% | 64.5% | 100% | 22.6% | 0% | ¥0.00 |
| 全云端 DeepSeek | 86.8% | 91.9% | 100% | 8.1% | 100% | ¥1.69 |
| 证据门控路由（本作品） | 85.3% | 93.5% | 100% | 3.2% | 39.7% | ¥0.66 |

表 1　三类策略在同一批题上的对比（n=68）

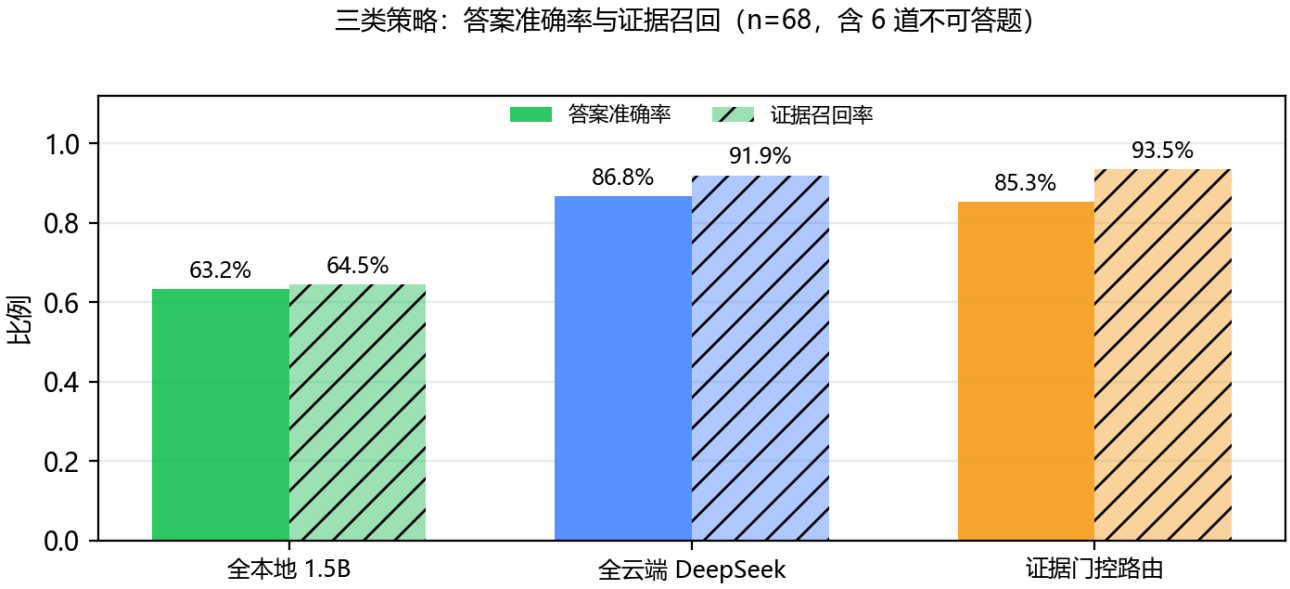



图 2　答案准确率与证据召回：门控路由证据召回最高，准确率与全云端只差 1 题

结论可以概括成一句话：门控路由用不到四成的云端调用，拿到了与全云端持平的准确率（差 1 题），同时把证据召回与误拒率做到最好，成本降 61%。

全部判分不使用大模型当裁判：数值题比数词、短答题比别名、拒答题看是否说了"原文未提及"。判分脚本、逐条结果与重判脚本都在仓库里，评委可以用一条命令复算每一个数字。

# 四、项目中涉及的关键技术

## 4.1 PDF 解析：页码、坐标与章节

用 pymupdf 抽取 block 级文本与 bbox，按行聚合到 300–500 字的片段，跨页强制断开，剔除页眉页脚与被半数以上页面重复的行，遇到"References"直接截断。每个片段保存行级坐标数组，这是前端精确高亮的前提。

## 4.2 检索：BM25 + 离线双语索引

目标部署环境是 2 vCPU / 8 GB、无 GPU，稠密向量建库成本过高，因此使用 BM25（k1=1.5，b=0.75），英文做轻量词干化、中文按字 bigram 切分。真正的难点是中文问、英文原文的词面失配：解法是建库期把每块译文并入索引（3 篇共 328 块，约 ¥0.34），运行期中文问句命中译文，锚点仍回指英文原文。62 道可答题上，正确页进 top-1 的比例 91.9%、top-5 96.8%。

## 4.3 证据锚点与三条一致性校验

受约束生成 + 确定性对齐产出锚点后，用三条确定性规则校验：引用编号必须存在、答案中的数字必须在被引片段中找到、英文术语必须在原文找到。三条规则都可被逐条复算，且能给出可统计的指标（引用有效率、数字一致率）。

## 4.4 校验门控与端云协同

本地与云端被抽象成同一个 `Provider` 协议，换模型不需要改编排代码；门控逻辑只有十行：本地先答 → 锚点校验 → 通过则保留、不通过则升级云端重答并重新校验。云端密钥缺失或调用失败时自动降级本地，并标注"未通过校验"而不是返回空结果。

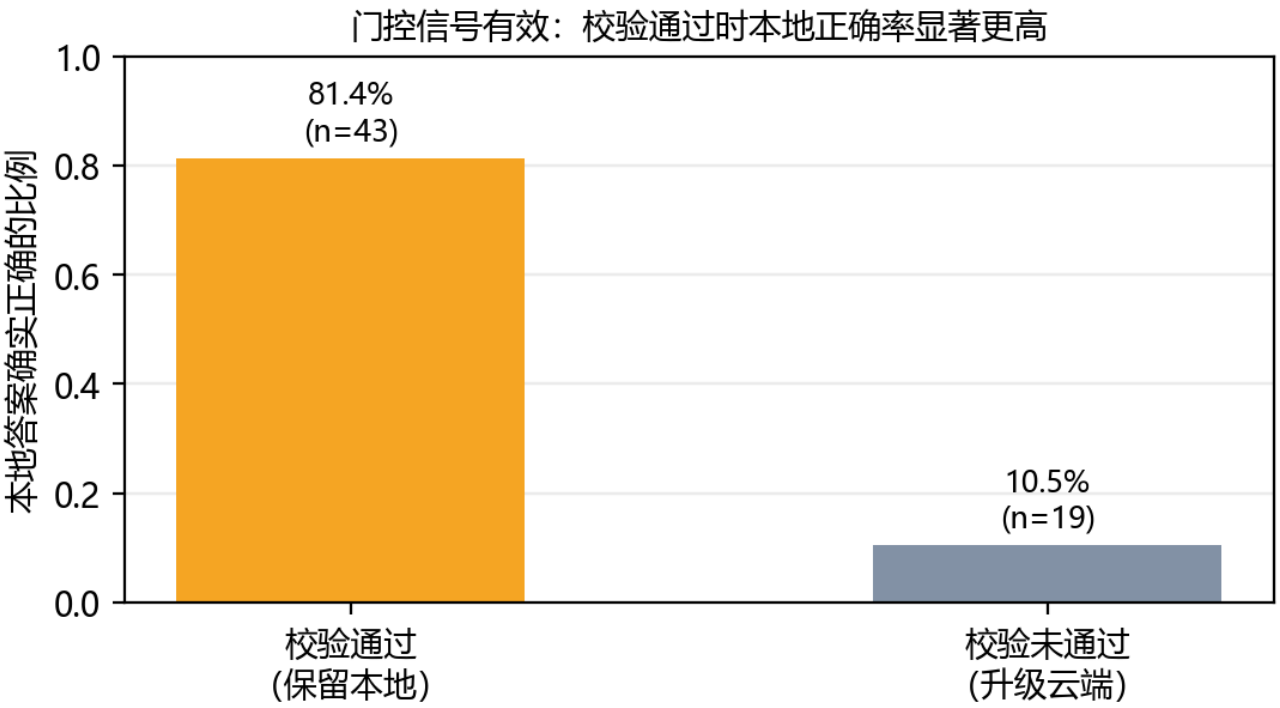



图 3　校验通过时本地答对率 81.4%，未通过时仅 10.5%——这就是门控的可操作空间

# 五、项目相关的图片

界面全貌（答案、引用角标、证据锚点、原文高亮、路由决策与账本）见 1.2 节的图 1；三类策略的准确率与证据召回见 2 节的图 2；门控信号的判别力见 4.4 节的图 3。

另附：成本—质量曲线（图 2 的对偶视角）与含答案的界面截图，均在仓库 `docs/figures/` 目录下，文件名分别为 `fig_paper_cost.png` 与 `ui_paper_answer.png`。

# 六、项目是否发表过或获奖

未发表、未获奖。本项目为 2026 第九届传智杯参赛作品，从选题、设计、实现到评测均在赛期内完成，代码仓库保留了按模块推进的完整提交历史。

# 七、是否在其他项目基础上优化

不是。本项目为独立开发，没有基于任何既有项目或往届作品修改。用到的第三方开源组件（pymupdf、FastAPI、scikit-learn、Ollama、Qwen2.5、llama.cpp）均为公开许可，用途仅限于推理框架与工具库，与本作品的核心方法（离线双语索引、确定性锚点对齐、证据门控路由）无直接对应关系。

需要说明的一点：本作品的第一版（预指导提交版本）解决的是"通用算术问答的端云路由"，评委建议把路由对象换成赛题场景。本版保留其评测方法论与工程骨架，把承重墙换成"论文精读 + 证据锚点"，并按实测结果把路由信号从"预测难度"改为"校验证据"。这一过程在技术文档 6.4 与 8.4 节有完整记录。

# 附：配套材料索引

| 材料 | 位置 |
| --- | --- |
| 技术文档（24 页） | docs/HybridMind_技术文档.pdf |
| 在线演示 | https://hanyue0-hybridmind.ms.show/paper |
| 演示视频 | docs/HybridMind_演示视频_最终版.mp4 |
| 源代码 | https://github.com/hanyue0/hybridmind |
| 评测数据与结果 | data/qa_paper.jsonl、data/results/ |